Both strings and tries are recursive structures, whose unit of iteration is the single character; each string, in fact, can be described as either:

- The empty string;
- A character followed by a 1-character-shorter string.

For instance, the string “home” is made of the character ‘h’ concatenated to the string “ome”, which in turn is ‘o’ + “me” etc…, until we get to “e”, which can be written as the character ‘e’ concatenated to the empty string “”.

A trie, on the other end, stores strings as paths from the root to key nodes; we can describe a trie T as a root node connected to (at most) |∑| shorter tries: if a sub-trie T’ is connected to the root by an edge marked with character c (c ∈ ∑), then for all strings s in T’, c + s belongs to T.

For instance, in figure 6.3, the root has only one outgoing edge, marked with ‘a’; considering T’ as the only sub-trie of the root, T’ contains the word “n”, and this means that T contains ‘a’ + “n” = “an”.

Since both strings and tries are recursive, it’s natural, therefore, to define the search method recursively; we can restrict to consider just the case where we search the first character of a string s=c+s’ starting from the root R of the a trie T: if c, the first character of s, matches an outgoing of R, then we can search s’ in the (sub)trie T’. We assume the root of the sub-trie is not null (as we'll see a few lines ahead, it's a reasonable assumption).